Use case: AL-4 Accounts & Roles

Epic. A borrower registers and signs in on the iOS app. A broker or underwriter signs in to the web portal and sees only what their role allows, so client files stay private.

Item	Detail
Epic	AL-4 Accounts & Roles
Feature (from the brief)	Mobile: Register/Login (email/password or social, Firebase Auth). Web: Role-Based Access (brokers see only their referred clients; underwriters see the whole portfolio and can sign off). Core: Centralised Data Store (user profiles, broker assignments).
Owner	Nikoloz (writes this use case, and owns the web stories AL-17 and AL-18). Ibrahima owns the mobile stories AL-16 and AL-142. Dumi owns the backend work under them: role claims and the broker-client rules.
Sprint	Sprint 1 (8 to 25 Oct 2026)
Wireframes	Web: AL-17/wireframe.png (sign in) and AL-18/wireframe.png (My Clients) in this folder. Mobile: Ibrahima's AL-16 and AL-142 folders. Figma: https://www.figma.com/design/F6lldwqy172ox5SeALdDEU

Goal

As a borrower, broker or underwriter, I want to sign in and see only what my role allows, so that client files stay private and each person gets the right tools.

Actors

- Borrower: uses the iOS app, registers, signs in and chooses a broker (and can change them later).
- Broker: signs in to the web portal and sees only the clients who chose them.
- Underwriter: signs in to the web portal and sees every application.
- Firebase Auth: signs people in (email and password, Google on mobile) and issues the ID token. The token carries a role claim: borrower, broker or underwriter.
- al-core: sets the role claims, stores user profiles and broker assignments in Firestore, and holds the Firestore rules that decide who may read what.

Preconditions

- The Firebase emulators (or project) are running with the seed: one underwriter, two brokers, borrowers, and applications assigned to each broker.
- Every user has exactly one role, set as a custom claim on their account by al-core. The role field on the user document is only a copy and is never trusted.
- The portal runs on a machine that can reach Firebase Auth and Firestore.

Trigger

A person opens the app or the portal and signs in.

Main flow

1. A new borrower registers in the iOS app with email and password or Google, then picks a broker from the list. The choice is saved against their account.
2. A broker or underwriter opens the web portal at /login and enters their email and password. No role is asked for.
3. Firebase Auth checks the sign-in details and returns an ID token. The portal reads the role claim from that verified token.

4. The portal sends a broker to /broker and an underwriter to /underwriter. Pages for the other role are not reachable.
5. A broker opens Clients and sees only the clients who chose them, and their applications. An underwriter opens the pipeline and sees every application.
6. When a borrower changes broker, the client leaves the old broker's list and joins the new one's, without a reload.
7. The person signs out. The portal reloads at /login and protected pages redirect to it.

Alternative flows

- A. Google sign-in. Borrowers can use Google on mobile. The web portal sign-in is email and password only in Sprint 1; Google and demo role buttons on the portal moved to Sprint 2.
- B. Token refresh. The portal checks the role again whenever Firebase refreshes the ID token. If the role changed to another valid role, the page reloads so access is decided again.
- C. A signed-in user opens /login. They are sent to their own portal home.

Exceptions

- Wrong email or password: a neutral error beside the form that does not say which part was wrong.
- A borrower signs in to the portal: refused with "This portal is for brokers and bank staff", with a Sign out button.
- A user with a missing or unknown role: "Not allowed" and a Sign out button.
- A broker opens an underwriter address (or the other way round): "Not allowed" with a link back to their own portal. None of the other role's content is shown.
- The ID token lookup fails (network or Firebase problem): "We could not verify your session, reload to try again" with a Retry button. This is not shown as a role refusal.
- A broker opens another broker's client by URL, or a client that does not exist: the same "not found" page. The page only looks in the broker's own list, which the Firestore rules limit to their clients.
- The profile query is refused or fails: clients are still listed, by client ID, with a notice that names could not be loaded.

Postconditions

- Success: the person is signed in and sees only the views and data for their role. The broker's list matches the Firestore rules.
- Failure: no protected content is shown, and the person is told what to do (sign in again, retry, or ask an administrator).

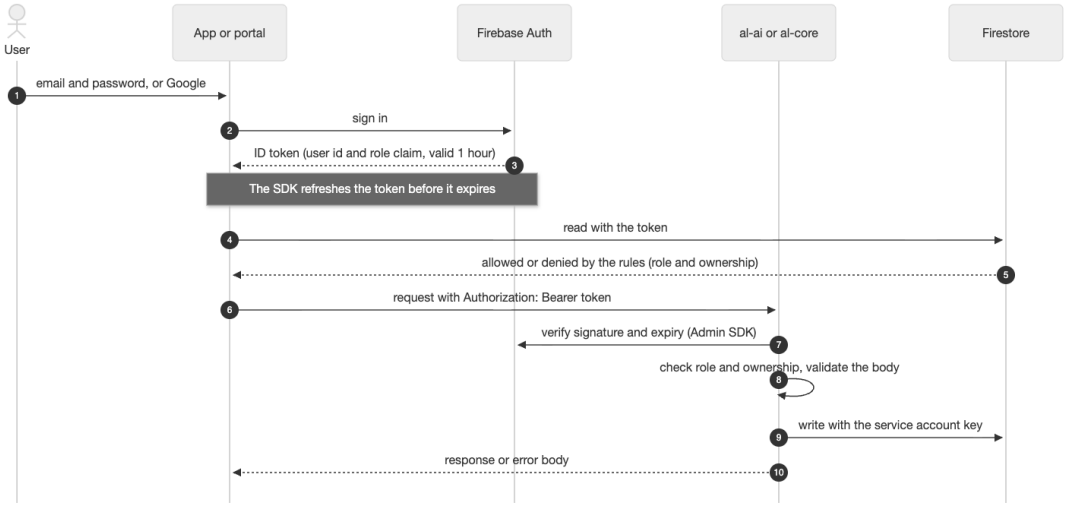
Business rules

- The role comes only from the verified role claim in the ID token. Browser storage, URL values, form fields and hidden buttons are never used to decide access.
- The Firestore rules are the data boundary. A broker's query has to filter on their own uid or the rules refuse it. The portal's route guard only decides what is shown.
- The route guard is not an API authorisation boundary. Any API the portal calls has to verify the ID token itself: 401 for a missing or invalid token and 403 for the wrong role. The portal makes no API calls in Sprint 1, so this is not yet exercised.
- A client is a borrower who has chosen this broker or made an application with them. Brokers never see another broker's clients.

- Underwriters see every application. The team decided that brokers do not see Draft applications (AL-263); that is built in Sprint 2.

Flow diagram

How sign-in and the role check work, from docs/diagrams/07-auth-sequence.png:



Stories in this epic

Story	What	Owner	Sprint	Status on 23 Oct
AL-16	A borrower registers and logs in with email/password or Google (iOS)	Ibrahima	Sprint 1	Done. Sign out from Your details moved to its own Sprint 2 story (AL-256)
AL-17	A broker or underwriter logs in to the web portal and sees only the views for their role	Nikoloz	Sprint 1	Done
AL-18	A broker sees only the files of clients they referred	Nikoloz	Sprint 1	Done
AL-142	A borrower chooses their broker after sign-up and can change them later (iOS)	Ibrahima	Sprint 1	Done

What Sprint 1 delivered

- Login page, role-based route guard and the "Not allowed" and borrower-refusal pages (AL-17, PR 28). A later fix made a role change on a token refresh reach the guard, and a failed token lookup show a retry page (AL-243, PR 76 and PR 96).
- The broker's client list and client file, filtered to the signed-in broker, live, with search and a status filter (AL-18, PR 68).
- Role custom claims and the broker-client assignment with Firestore rules and a two-broker seed (Dumi: AL-55 and AL-56).
- Borrower registration, sign-in and broker choice on iOS (Ibrahima: AL-16 and AL-142, see his folders).

Epic done when (from Jira)

- A use case for this feature is written in Confluence and linked here: this document.
- Every story in this epic meets its acceptance criteria and is merged to main. All four stories (AL-16, AL-17, AL-18 and AL-142) are Done. The one piece left out of AL-16, sign out from the iOS Your details screen, is its own Sprint 2 story (AL-256).

- On the seeded data: a new borrower can register, sign in and choose a broker (checked on the emulators by Ibrahima); the seeded broker sees only their own clients and the second broker sees a different list (checked, AL-18); the seeded underwriter sees every application (checked: 7 applications in the queue on the emulators).

Notes

- Test cases: AL-17/test-case.docx and AL-18/test-case.docx in this folder, and Ibrahima's AL-16 and AL-142 test cases in his folder.
- Moved to Sprint 2 and recorded on the tickets: the split sign-in layout, Google and demo role buttons on the portal, 401 and 403 handling once the portal calls an API, and messaging the borrower from the client file.